Removing proof layers from the certified 215 state construction

Follow-up research report

21 October 2026

Abstract

The strongest new simplification is structural: the two actual source decoder trees used by K215 normalize to exactly the same 1,426-vertex increment-weighted test graph. This could replace the entire explicit 2,240-pair source correspondence with one generic adequacy theorem and a graph-equality check, rather than merely deleting its reverse checks. A second concrete patch removes two unnecessary syntactic round-trip inductions from the logical calibration by using an existing semantic round trip. A finite source-DAG and tape-window product also factors repeated internal decoder traversal. Startup sweep compression is real but secondary; exact runtime-result memoization is negligible. These are independently checked data and uncompiled formalization proposals. No new proof-assistant endpoint, measured kernel speedup, or smaller Turing machine is claimed.

Current upstream status. A later read-only review found main at `05d063a`, twelve commits beyond the inspected snapshot. The earlier symmetry refactor has already been adopted in the maintained source with recorded kernel checks; it is not a new unimplemented opportunity in this report. The two source trees and all sixteen forward blocks are textually unchanged, so the new graph result still applies. The inspected runtime and model modules are also unchanged apart from an extra terminal newline in older saved copies. The exact current pin and comparison receipts are packaged; this report does not claim a new local proof-assistant build.

1 Replace the correspondence with graph equality

At the pinned source snapshot

`c9dc79323429112c698100f3226a6e4d56c8b782`

the actual Lean trees `KSource215.t215` and `KSource215.tD` each have 4,751 primitive leaves. Enumerate those leaves from left to right. Every decrement-test leaf and the halt leaf becomes a checkpoint labelled by its leaf ordinal. Collapse intervening increments and structural control steps into a directed edge carrying an increment-count vector.

Independent parsing and complete edge evaluation establish:

- Both graphs have 1,425 decrement vertices and one halt vertex, hence 1,426 vertices and 2,850 zero/nonzero edges.
- Every vertex label, both successor ordinals and every increment vector agree exactly. Initial segments also agree and are empty.
- All 2,240 old pointer pairs normalize to the same ordinal on their two sides; only 1,426 distinct ordinals occur. The remaining 814 records are pointer aliases, not distinct semantic checkpoints.

- Each collapsed segment takes at most five source transitions. The sum over all 2,850 edge segments is 3,420 on each side. Terminal inspection requires at least six units of the current segment evaluator's fuel.

This is identity on independently computed ordinals, not a guessed graph isomorphism. The proposed final source proof is: compile both finite trees to weighted graphs, kernel-check equality of the complete graphs, then apply one generic graph-adequacy theorem. A Python equality test or a matching hash is not a replacement for that theorem.

1.1 The exact canonicalization lemma

The existing tree lookup traverses with implicit trailing zero bits and ignores any unused pointer suffix after reaching a primitive leaf. Its returned path Q is the full consumed root-to-leaf path, which can differ from the input pointer P. Define $\mathrm{canon}_T(P)$ as the ordinal of Q when lookup succeeds.

First prove lookup idempotence at the consumed path: if lookup of P returns (e, Q), lookup of Q returns the same (e, Q). The existing single-step theorem executes instruction e at Q, not blindly at P. Thus both pointers have the same halt test and next-step semantics for every register file. This is a semantic quotient of control pointers; it does not ignore enclosing ancestors or identify arbitrary source subtrees.

A fuelled segment evaluator performs only increments, breaks and continues until reaching a decrement or halt. It rejects failed lookup, fuel exhaustion and malformed instructions. It records the terminal canonical ordinal and a finite vector of register increments. No decrement or register test occurs inside a successful segment, so increments commute and the vector determines its action for all natural-valued registers.

At a decrement of register r, the zero edge starts from the exact break target and leaves registers unchanged before the segment. The nonzero edge first decrements register r, then follows the increment-path target and adds the segment vector. Compile both outcomes of every vertex, including outcomes that seem unreachable in a finite test. A halt vertex has no outgoing edge. Include the segment from the empty pointer as the entry descriptor.

1.2 Generic adequacy without another simulation framework

For a successful compilation $T \mapsto G$, every graph edge expands to one actual decrement step followed by its finite segment. Conversely, any source execution at a checkpoint must take exactly that branch and segment. There is no hidden test or halt inside the segment. Each nonterminal graph step consumes at least one source transition, even when the following segment is empty; the initial segment may be empty. Fuel exhaustion rejects a control-only loop instead of silently collapsing it.

The smallest integration can reuse existing `RMBisim.Bisim`: relate pointers when their canonical checkpoint ordinals agree. Complete graph equality supplies its initial and step fields using the existing segment lemmas. Then invoke the already proved symmetric halting theorem. This avoids developing a second graph operational semantics and preserves the existing source theorem for arbitrary register files and all downstream endpoints.

1.3 Why simple deduplication is wrong

The current `seg` returns a *raw* terminal pointer. Existing branch checkers compare that pointer literally with certificate pointers; they do not canonicalize it first. Of 5,700 endpoint-side comparisons, 5,324

require normalization. Therefore the generated 1,426-entry canonical table does **not** pass the unchanged checker. It is data for a proposed normalization-aware checker, not a finished replacement certificate.

The graph approach can ultimately remove even that smaller table: compute ordinals from both trees instead of storing a correspondence. It reduces semantic checkpoint cases by 36.34%, but still requires kernel evaluation of both graph constructions and equality. Historical forward-check cost was about 1,087.66 summed compiler-seconds; no saving for the new design has been measured. Long reference-tree traversal and allocation can still dominate.

The primitive words themselves differ at 855 positions, including adjusted continue depths and commuting increments. Unrestricted tree associativity is false because breaks and continues depend on context. Graph compilation resolves those contextual targets before comparing them, avoiding a large family of risky local rewrite rules.

2 Remove unnecessary syntax from the model argument

The concrete proposed change to `KSeedModel222.lean` removes the inductive lemmas `enc_dec_enc` and `eqFree_dec_enc` from the endpoint's local dependency chain. Rather than first deriving the decoded numerical bundle, derive the original named bundle directly using the existing equality-free interpretation theorem and the bundle encoding identity. The model hypothesis already ranges over all such derivable named formulas.

At the final call use the imported theorem

$$\mathrm{Sat}(\mathrm{dec}(\mathrm{enc}\, f)) \iff \mathrm{Sat}(f)$$

in an irreflexive membership structure. This provides exactly the semantic fact required by the lower-bound argument. It does not assume the generally false syntactic identity $\mathrm{dec}(\mathrm{enc}\, f) = f$: encoded falsity decodes to self-membership, whose semantics is false only under the required irreflexivity assumption.

The supplied exact patch removes 28 lines of two induction proofs, replaces the local bundle derivation by a direct named-formula theorem, and adds three call-site lines: a net 25-line reduction, including the corrected adjacent comment. The final theorem statement is unchanged. The patch is **uncompiled**; integration must search for external uses of the deleted public lemmas, or leave compatibility lemmas while removing their endpoint dependencies. This does not delete the separate freshness proof for a bundle's encoded matrix.

2.1 Two reusable model interfaces

A 47-line uncompiled `SeedTransport.lean` proposes pointwise relation congruence and transport along a surjective map that preserves and reflects both membership and the selected binary relation. Surjectivity is essential for lifting arbitrary target witnesses and subset quantifiers. No injectivity or code well-formedness is assumed. The generic theorem can replace repeated witness reconstruction, but it cannot replace the quotient's surjectivity, semantic interpretation or relation-preservation proofs.

The two directions of bundle semantics can also share a single normal-form equivalence for distinct parameter roles. Prove request and closure semantics once, normalize the existential set and root conditions, and reuse the resulting equivalence. Keep the existing $a = b$ forward-only degenerate case separate: collapsing two roles changes variable updates, so the ordinary-role equivalence cannot simply be extended to it. Freshness and the truth-code alias remain genuine obligations.

2.2 Share logical rule interfaces without strengthening them

Repeated propositional and Leibniz arguments can be parameterized by a derivability predicate and its explicit rules. An admissibility flag distinguishes unrestricted formulas from equality-free formulas; a separate binder-safety predicate records which quantified substitution cases are actually available. Instantiate the restricted calculus with no quantified substitution case rather than silently granting it unrestricted specialization. Begin with the small propositional interface, then atomic transport, then the guarded structural induction.

This is a consolidation plan, not a proved line-count saving after interface declarations. Lean's semantic quotient chain and Rocq's named/de-Bruijn route are not identical. A dependency deletion in one cannot be ported to the other merely because theorem names look similar. The all-natural numerical-calculus consistency calibration remains the mathematical content, not bureaucracy that an abstraction may replace with an axiom.

3 Factor the internal decoder by a finite product

A second substantial opportunity is now independently checked for three pinned tables: the exact certified K215 baseline, ST215 and GC–ZC215. All 430 rows of the current K215 input match the literal formal `KTable215.lean` table under native relabeling $301 \mapsto 102$, $302 \mapsto 120$ and halt $254 \mapsto 230$, with start zero fixed. The full 4,751-site source list also matches the formal source tree. A separate dense-table comparison agrees. It factors repeated traversal of internal source-decoder nodes into local tape templates plus a finite product of the source DAG and physical control context. It does not depend on selected register samples.

Each local template starts at the final cell of an explicit binary word of length one to three. Every literal machine transition must read inside that window; the checker rejects before either opaque exterior is read. The complete window must be restored, the head must end immediately to its right, and the exit state must agree. Induction on this finite trace therefore frames arbitrary exterior tape contents. At most two preceding bits suffice for the templates proved here; no globally minimal history claim is made.

The product key is exactly

$$(\text{source DAG node},\ \text{physical state},\ \text{last up-to-two bits}).$$

Physical state alone does not identify a source node. The checker rebuilds the source DAG from the complete pinned site list, checks both branches at every reachable product state, and verifies their actual source children. Child DAG indices strictly decrease. History updates retain the final two bits. The root begins with only the single left marker bit one, so no initial template may assume a nonexistent second history bit.

Machine	Templates	Product states	Edges	Leaf contexts
K215	281	234	468	97
ST215	281	235	470	95
GC–ZC215	282	236	472	95

Induction on the DAG proves coverage of every valid source-code path, with exact head alignment and preservation of the whole code and exterior. Source depth is checked at most 48, below the 64-cell

header bound. Trailing padding, header, arbitrary gap, registers and spare list remain exterior during these macros.

The expanded paths contain 155,554, 141,307 and 155,560 uses of internal decoder edges respectively. These are repeated internal-edge uses, not whole primitive guards. The certificate does **not** yet prove leaf operations, register engines, pointer carry/strip returns, source semantics or consistency calibration. Its 97/95/95 exposed leaf contexts form the interface for a later theorem. The packet retains the older, differently labelled K215-family input as separately named historical evidence; the report's K215 counts come from a fresh run on the aligned current table. Port the local opaque-window lemma and finite product checker to the proof assistant before replacing any certified runtime modules.

3.1 Measurements that rule out a misleading shortcut

Exact symbolic-state memoization saves only 295 of 2,442,801 steps of the aligned K215 baseline, about 0.012%; ST215 saves 269 of 2,298,318 and GC–ZC215 saves 358 of 2,442,871. Every complete event trace is distinct. These census steps come from an owned transcription of the inspected Lean executor with its existing box-lemma oracle, not the differently counted earlier register-macro replay. The box theorems remain assumptions to be discharged by the existing formal development.

The executor already short-circuits by state and head before normalizing endpoint comparisons: K215 performs 15,675 such comparisons, not 2.4 million. A comparison can normalize multiple lists; 15,675 is not a count of individual normalization-function calls. Counted execution would reduce these comparisons to 6,176, while reducing all cheap-plus-expensive target tests from 2,442,801 to 6,176. A proposed counted executor can use an untrusted exact step count and check the endpoint once, with a soundness induction over the existing step theorem and a positive-progress flag. It is an uncompiled, modest interface simplification, not evidence of a hundreds-fold speedup. Reusable framed decoder semantics is the stronger opportunity.

4 Startup compression is real but secondary

The certified K215 startup uses 13 computational chunks across 14 modules, not 14 computational chunks. It already has balanced transition lookup, a zipper, balanced block execution, generic dense/sparse relabeling and arbitrary-length same-state scan lemmas. Reimplementing those abstractions would duplicate work.

An owned generator compresses its 95,714 primitive transitions into 39,889 macro operations, including 3,406 sweeps longer than one. An independent absolute-coordinate tape verifier checks every read interval, table row, self-loop condition, write and displacement, and agrees with the generator's separate list zipper. The saving is 55,825 transition dispatches, or 58.3%, *not* a demonstrated 58.3% reduction in kernel time.

The exact 16,384-step published checkpoint and final complete tape agree. The final dense state 83 is sparse state 84, at absolute head -8, with 95 one cells. Its full right-side list has length 208, with the specified 65-cell gap, sentinel, nine boot registers and forty spare registers of value one; the entire exterior is zero. Five mutated certificate fields are rejected.

The proposed formal route uses positive-length runs, proves expansion to the existing complete zipper, and reuses the scan lemmas. Empty-side blank extension and run coalescing need explicit proofs. The uncompiled `SweepGlue.lean` assumes the missing concrete run-length refinement; it is not a finished startup certificate. The endpoint needs reachability. Retaining the exact 95,714-step statement additionally requires counted scan theorems, not merely reachability lemmas.

Startup accounted for only about 111 of 4,500 historical summed compiler-seconds, roughly 2.5%. Its main value is fewer generated chunks and reuse of existing scan proofs, not a large total-build speedup.

5 Reproduction and formalization order

The portable packet preserves pinned Lean source data, owned independent parsers and checkers, exact graph and startup receipts, proposed patches, and explicit proof boundaries. All formal prototypes remain uncompiled. No new machine, altered consistency strength, or 214-state claim is made.

Run from the extracted packet with a fresh external directory:

```
python -I -B verify.py --work /tmp/layer-check
```

The verifier regenerates the complete weighted graphs, canonical aliases and startup certificate in that directory, checks exact equality with archived outputs, checks all three framed decoder products, and checks input hashes. Assertions are mandatory. This is independent Python validation, not a kernel proof.

Prioritize: first implement the lookup-canonicalization lemma and graph-equality-to-bisimulation theorem, preserving the existing endpoint statement; next compile the small semantic round-trip patch; then benchmark internal decoder factoring and startup scans. Keep old and new endpoint axiom reports, clean build timings and memory measurements. Delete generated proof modules only after the replacement checks compile and preserve the final assumptions.